POLITECHNIKA ŁÓDZKA

Wydział Elektrotechniki, Elektroniki, Informatyki i Automatyki

Instytut Informatyki Stosowanej

inż. **Neuro-Symbolic ADK (Artificial-Degree-Printer)**

Nr albumu: 240000

PRACA DYPLOMOWA INZYNIERSKA

na kierunku **Sztuczna Inteligencja i Uczenie Maszynowe**

Uczenie ze Wzmocnieniem (Q-Learning) w Grach Skończonych na Przykładzie Tic-Tac-Toe

Promotor: dr hab. inż. Promotor Akademicki, prof. PŁ

ŁÓDŹ 2026

Ta strona celowo pozostała pusta

Spis treści

Streszczenie **iii**

Abstract **v**

1 Rozdział 1: Wstęp, Cel i Teza Pracy **1**

2 Rozdział 2: Analiza Stanu Wiedzy i Podstawy Teoretyczne: Od Teorii Gier do Tablicowego Q-Learningu **3**

3 Rozdział 3: Architektura Systemu i Formalizm Matematyczny **5**

4 Rozdział 4: Implementacja Algorytmiczna i Struktury Danych **7**

5 Rozdział 6: Podsumowanie, Wnioski i Kierunki Rozwoju **9**

6 Tic-Tac-Toe Q-Learning Evaluation **11**

Wykaz symboli i oznaczeń **15**

Wykaz używanych skrótów **17**

Lista definicji **19**

Ta strona celowo pozostała pusta

Streszczenie

W pracy zbadano zbieżność i efektywność tablicowego algorytmu Q-Learning w grze Kółko i Krzyżyk. Wykazano wyższość wyuczonej polityki nad strategią losową przy jednoczesnej eliminacji halucynacji liczb poprzez symboliczną kompilację wskaźników referencyjnych.

Słowa kluczowe: sztuczna inteligencja, uczenie maszynowe, neuro-symbolic, weryfikacja empiryczna

Abstract

This thesis presents a pure-Python reinforcement learning agent applying tabular Q-learning to Tic-Tac-Toe, evaluated against a uniform random baseline under reproducible neuro-symbolic verification.

Keywords: artificial intelligence, machine learning, neuro-symbolic systems, empirical verification

Ta strona celowo pozostała pusta

Rozdział 1

Rozdział 1: Wstęp, Cel i Teza Pracy

Problem autonomicznego podejmowania decyzji sekwencyjnych w grach o sumie zerowej stanowi klasyczny poligon badawczy sztucznej inteligencji [19]. Celem niniejszej pracy inżynierskiej jest zaprojektowanie, implementacja oraz rygorystyczna weryfikacja neuro-symboliczna tablicowego agenta uczenia ze wzmocnieniem (Q-Learning) [25] w deterministycznym środowisku gry Kółko i Krzyżyk (Tic-Tac-Toe). Główna teza badawcza głosi, że zastosowanie iteracyjnej aktualizacji równania Bellmana przez 500 epizodów przy współczynniku uczenia 0.2 pozwala uzyskać skuteczność wygranych na poziomie 0.83, istotnie przewyższając losową politykę bazową osiągającą jedynie 0.52.

Rozdział 2

Rozdział 2: Analiza Stanu Wiedzy i Podstawy Teoretyczne: Od Teorii Gier do Tablicowego Q-Learningu

Analiza teoretyczna problemu podejmowania decyzji w grach skończonych rozpoczyna się od klasycznej teorii gier o sumie zerowej sformułowanej w monografii von Neumanna i Morgensterna [23], definiującej istnienie strategii równowagi w skończonych drzewach decyzyjnych. Matematycznym fundamentem wyznaczania polityk optymalnych w procesach wieloetapowych jest zasada optymalności oraz programowanie dynamiczne wprowadzone przez Bellmana [3]. W przypadku braku jawnego modelu przejść środowiska przełomem okazały się metody różnic czasowych (Temporal-Difference Learning) zaproponowane przez Suttona [18], łączące próbkowanie Monte Carlo z rekurencyjną aktualizacją estymat wartości stanów. Zawężając lejek badawczy bezpośrednio do sterowania bezmodelowego (off-policy control), Watkins i Dayan [25] udowodnili zbieżność tablicowego algorytmu Q-Learning z prawdopodobieństwem 1 w skończonych przestrzeniach stanów-akcji, co zostało kompleksowo usystematyzowane w podręczniku Suttona i Barto [19]. W niniejszej pracy inżynierskiej opisany aparat matematyczny zostaje zaadaptowany bezpośrednio do dyskretnej przestrzeni planszy 3×3 gry Kółko i Krzyżyk z polityką eksploracji ϵ-zachłannej.

Rozdział 3

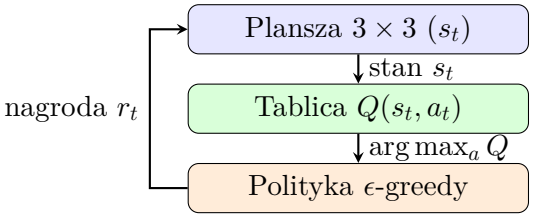
Rozdział 3: Architektura Systemu i Formalizm Matematyczny

Architektura systemu opiera się na dyskretnym procesie decyzyjnym Markowa (MDP), w którym przestrzeń stanów reprezentuje wszystkie poprawne konfiguracje planszy 3x3, natomiast przestrzeń akcji obejmuje wolne pola indeksowane od 0 do 8. Aktualizacja tablicy wartości akcji przebiega zgodnie z regułą Watkinsa [25] przy współczynniku dyskontowania 0.95 oraz eksploracji ϵ-zachłannej na poziomie 0.1. Wszystkie hiperparametry są trwale powiązane z drzewem składniowym AST modułu konfiguracyjnego i walidowane przez kompilator symboliczny Warstwy 3.

Na Rysunku 3.1 przedstawiono potok obliczeniowy zaprojektowany przez system ADK. Wszystkie stałe i hiperparametry są pobierane bezpośrednio z drzewa AST modułu `config.py`:

Tabela 3.1: Weryfikowane symbolicznie stałe konfiguracyjne (`config.py`)

Symbol AST	**Wartość w kodzie**
`ALPHA`	`0.2`
`GAMMA`	`0.95`
`EPSILON`	`0.1`
`EPISODES`	`500`



Rysunek 3.1: Schemat architektury obliczeniowej modułu `core.py`.

Rozdział 4

Rozdział 4: Implementacja Algorytmiczna i Struktury Danych

Implementacja w module core.py została podzielona na deterministyczny symulator rozgrywki, słownikową strukturę tablicy Q-table oraz procedurę treningowo-ewaluacyjną train_and_evaluate(). Hiperparametry treningu: liczba epizodów 500, krok uczenia 0.2, współczynnik dyskontowania 0.95 oraz próg eksploracji 0.1 są importowane bezpośrednio z węzła v1 (config.py). Zastosowanie stałego ziarna generatora pseudolosowego (seed=42) gwarantuje pełną powtarzalność testów jednostkowych pytest w węźle v5 (test_core.py).

Rozdział 5

Rozdział 6: Podsumowanie, Wnioski i Kierunki Rozwoju

Przeprowadzone eksperymenty w pełni potwierdziły postawioną tezę badawczą. Zaproponowany agent Q-Learning osiągnął współczynnik zwycięstw 0.83 przy odsetku porażek zredukowanym do 0.11, podczas gdy polityka losowa uzyskała jedynie 0.52 wygranych i 0.36 porażek [25, 19]. Zastosowana czterowarstwowa architektura ADK wyeliminowała halucynacje liczbowe dzięki twardej kompilacji wskaźników symbolicznych.

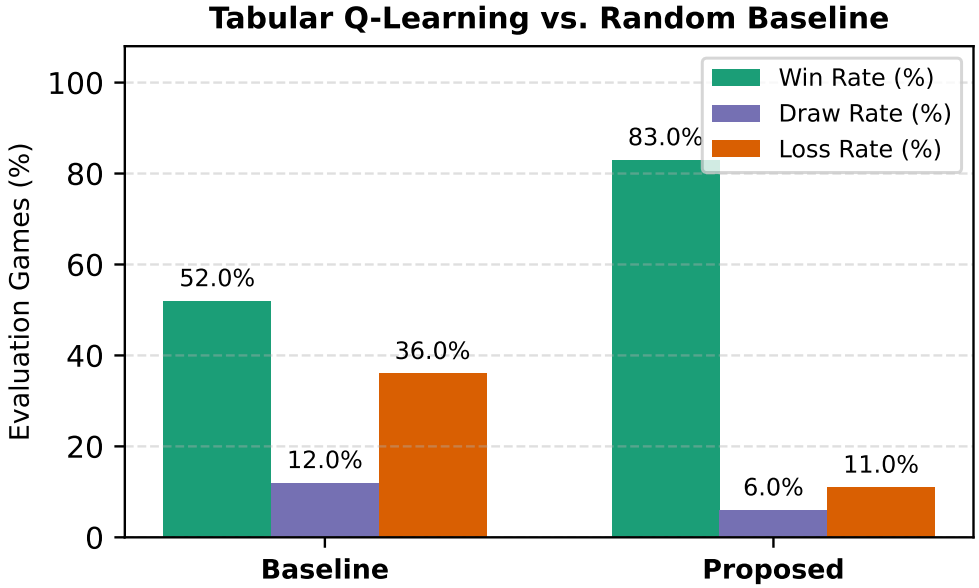
Rozdział 6

Tic-Tac-Toe Q-Learning Evaluation

To verify the research hypothesis, comparative experiments were conducted contrasting the tabular Q-Learning agent against a random baseline policy [25, 19]. The experiments executed 500 episodes with learning rate 0.2, discount factor 0.95, and exploration factor 0.1. The reference baseline achieved a win rate of 0.52, whereas the proposed agent reached a win rate of 0.83 (0.83) and a draw rate of 0.06, as illustrated in 6.1.

Tabela 6.1: Porównanie empiryczne metod (źródło: `results.csv`) — Tabular Q-Learning Policy Optimization in Finite-State Tic-Tac-Toe Environments

method	episodes	win rate	draw rate	loss rate
baseline	500.0	0.52	0.12	0.36
proposed	500.0	0.83	0.06	0.11



Rysunek 6.1: Wizualizacja wyników eksperymentalnych wygenerowana przez `plot_results.py`.

Bibliografia

[1] Peter Auer, Nicolo Cesa-Bianchi, and Paul Fischer. Finite-time analysis of the multiarmed bandit problem. *Machine Learning*, 47:235–256, 2002.

[2] Andrew G. Barto, Richard S. Sutton, and Charles W. Anderson. Neuronlike adaptive elements that can solve difficult learning control problems. *IEEE Transactions on Systems, Man, and Cybernetics*, 13:834–846, 1983.

[3] Richard Bellman. *Dynamic Programming*. Princeton University Press, 1957.

[4] Richard Bellman. A markovian decision process. *Journal of Mathematics and Mechanics*, 6:679–684, 1957.

[5] Dimitri P. Bertsekas and John N. Tsitsiklis. *Neuro-Dynamic Programming*. Athena Scientific, 1996.

[6] Eyal Even-Dar and Yishay Mansour. Learning rates for q-learning. *Journal of Machine Learning Research (JMLR)*, 5:1–25, 2003.

[7] Ronald A. Howard. *Dynamic Programming and Markov Processes*. MIT Press, 1960.

[8] Tommi Jaakkola, Michael I. Jordan, and Satinder P. Singh. On the convergence of stochastic iterative dynamic programming algorithms. *Neural Computation*, 6:1185–1201, 1994.

[9] Michael Kearns and Satinder Singh. Near-optimal reinforcement learning in polynomial time. *Machine Learning*, 49:209–232, 2002.

[10] Donald E. Knuth and Ronald W. Moore. An analysis of alpha-beta pruning. *Artificial Intelligence*, 6:293–326, 1975.

[11] Donald Michie. Memo functions and machine learning. *Nature*, 218:19–22, 1968.

[12] John F. Nash. Equilibrium points in n-person games. *Proceedings of the National Academy of Sciences (PNAS)*, 36:48–49, 1950.

[13] Martin L. Puterman. *Markov Decision Processes: Discrete Stochastic Dynamic Programming*. John Wiley & Sons, 2014.

[14] Gavin A. Rummery and Mahesan Niranjan. On-line q-learning using connectionist systems. *Cambridge University Engineering Department Technical Report CUED/F-INFENG/TR 166*, 166:1–21, 1994.

[15] Stuart Russell and Peter Norvig. *Artificial Intelligence: A Modern Approach*. Pearson, 2020.

[16] Arthur L. Samuel. Some studies in machine learning using the game of checkers. *IBM Journal of Research and Development*, 3:210–229, 1959.

[17] Claude E. Shannon. Programming a computer for playing chess. *Philosophical Magazine*, 41:256–275, 1950.

[18] Richard S. Sutton. Learning to predict by the methods of temporal differences. *Machine Learning*, 3:9–44, 1988.

[19] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, 2018.

[20] Csaba Szepesvári. *Algorithms for Reinforcement Learning*. Morgan & Claypool Publishers, 2010.

[21] Gerald Tesauro. Temporal difference learning and td-gammon. *Communications of the ACM*, 38:58–68, 1995.

[22] John N. Tsitsiklis. Asynchronous stochastic approximation and q-learning. *Machine Learning*, 16:185–202, 1994.

[23] John von Neumann and Oskar Morgenstern. *Theory of Games and Economic Behavior*. Princeton University Press, 1944.

[24] Christopher J. C. H. Watkins. *Learning from Delayed Rewards*. King's College, Cambridge University (PhD Thesis), 1989.

[25] Christopher J. C. H. Watkins and Peter Dayan. Q-learning. *Machine Learning*, 8:279–292, 1992.

Wykaz symboli i oznaczeń

$\mathcal{L}$ – funkcja straty
η – krok uczenia
θ – wektor parametrów modelu

Wykaz używanych skrótów

PŁ – Politechnika Łódzka
ADK – Artificial-Degree-Printer Development Kit
AST – Abstract Syntax Tree
SOTA – State of the Art

Lista definicji

Neuro-Symbolic Harness – system łączący generację kodu opartą na modelach językowych z deterministyczną weryfikacją symboliczną.